

COIN CHANGE PROBLEM-TEST CASES

Team ID- T 05

Table of Contents

1. Introduction / Scope	1
2. Test environment	2
3. Test Case Template	2
4. Test Cases	4
4.1 Basic Cases	4
4.2 Impossible Cases	5
4.3 Cases Where Greedy Algorithm Fails	6
4.4 Large Values/ Performance Cases	7
4.5 Edge / Boundary Cases	9
5. Summary Table	10
6. Conclusion	10
7. Appendices	11

1. Introduction / Scope

- Purpose: To document test cases for the coin change problem (minimum coins required)

- Scope: The test cases will cover normal, edge, and performance scenarios.

- Assumptions / Constraints:

1) coin denominations are positive integer

2) amount is a nonnegative integer.

3) Denomination set should be non empty

2. Test environment

Language	Python 3.13 (64 bit)
Interpreter	C Python
Operating System	Windows 11
Hardware	ASUS Vivobook 15
Processor	Intel Core i3-1315U
RAM	8 GB
External Services	None

3. Test Case Template

Below is table you should use for each test case.

Field	Description
Test Case ID	Unique Identifier(eg.TC001)
Scenario	What scenario is tested
Input	The coins array and target amount
Expected output	What the correct output should be
Actual output	Output given by code
Time	Time taken to execute the code in seconds
Status	Pass/Fail
Notes	Additional Observations

4. Test Cases

4.1 Basic Cases

Test Case ID: TC001

Scenario / Description: amount = 0

Input: coins = [1, 2, 5], amount = 0

Expected Output / Result: 0 (as no coin is required)

Actual Output: 0

Time: 1.22×10^{-5} (approximately)

Status: Pass

Notes : Edge case; ensure zero is handled.

Test Case ID: TC002

Scenario / Description: single coin equals amount

Input: coins = [7], amount = 7

Expected Output: 1 (one coin)

Actual Output: 1

Time: 5.02×10^{-5}

Status: Pass

Test Case ID: TC003

Scenario / Description: single coin equals amount

Input: coins = [7,10,30], amount = 30

Expected Output: 1 (one coin)

Actual Output: 1

Time: 4.71×10^{-5}

Status: Pass

4.2 Impossible Cases

Test Case ID: TC010

Scenario / Description: no coins available

Input: coins = [], amount = 5

Expected Output: Not well defined

Actual Output: Error, None

Time: 0.01

Status: Pass

Notes: Denomination set must be non empty

Test Case ID: TC011

Scenario / Description: Negative Amount

Input: coins = [4,10], amount = -2

Expected Output: Not well defined

Actual Output: Error, None

Time:0.07

Status: Pass

Test Case ID: TC012

Scenario / Description: Non positive denomination

Input: coins = [0,1,4], amount = 9

Expected Output: Not defined

Actual Output: Error, None

Time: 0.07

Status: Pass

Notes: Denomination set must be positive (assumption)

Test Case ID: TC013

Scenario / Description: Non positive denomination

Input: coins = [-2,2,4], amount = 10

Expected Output: Not well defined

Actual Output: Error, None

Time:0.05

Status: Pass

Notes: Denomination set is of positive integers

4.3 Cases Where Greedy Algorithm Fails

Test Case ID: TC020

Scenario / Description: Greedy fails

Input: coins = [1, 4, 9], amount = 12

Expected Output: 3(4+4+4)

Greedy Algorithm Output: 4 (9+1+1+1)

Actual Output: 3

Time: 3.62×10^{-5}

Status: Pass

Notes: This ensures the algorithm is not purely greedy.

Test Case ID: TC021

Scenario / Description: Greedy fails

Input: coins = [7, 9], amount = 14

Expected Output: 2 (7+7)

Greedy Algorithm Output: -1 (Remaining =5)

Actual Output: 2

Time: 2.24×10^{-5}

Status: Pass

Notes: Works in cases where greedy fails

Test Case ID: TC022

Scenario / Description: Greedy fails

Input: coins = [27,30], amount = 81

Expected Output: 3(27+27+27)

Greedy Algorithm Output: -1 (Remaining =21)

Actual Output: 3

Time: 0.0001

Status: Pass

4.4 Large Values/ Performance Cases

Test Case ID: TC040

Scenario / Description: large amount performance

Input: coins = [1, 5, 10, 25], amount = 10^6)

Expected Output: 40000(25×40000)

Actual Output: 40000

Time:0.614

Status: Pass

Notes: Test time / memory usage.

Test Case ID: TC041

Scenario / Description: large amount performance

Input: coins = [1, 5, 10, 25], amount = 10^7

Expected Output: 400000(25×400000)

Actual Output: 400000

Time:6.533

Status: Pass

Test Case ID: TC042

Scenario / Description: large amount performance

Input: coins = [1, 5, 10, 25], amount = 10^8

Expected Output: 4×10^6 ($25 \times 4 \times 10^6 = 10^8$)

Actual Output: 4×10^6

Time:150.42

Status: Pass

Test Case ID: TC043.

Scenario / Description: large amount performance

Input: coins = [1, 5, 10, 25], amount = 5×10^8

Expected Output: 2×10^7

Actual Output: Error

Time:354.30

Status: Fail

Test Case ID: TC044

Scenario / Description: Large denomination, small amount

Input: coins = $[10^5, 3 \times 10^6, 10^7, 4 \times 10^5]$, amount = 10^6

Expected Output: -1

Actual Output: -1

Time:0.0005

Status: Pass

4.5 Edge / Boundary Cases

Test Case ID: TC050

Scenario / Description: coin denominations with duplicates

Input: coins = [1, 2, 2, 5], amount = 7

Expected Output: 2

Actual Output: 2

Time: 7.24×10^{-5}

Status: Pass

Notes: Ensure duplicates don't break logic.

Test Case ID: TC051

Scenario / Description: coin denominations with duplicates

Input: coins = [10, 12, 12, 20], amount = 34

Expected Output: 3

Actual Output: 3

Time: 5.17×10^{-5}

Status: Pass

5. Summary Table

Category	# of Test Cases	Test ID	# Cases Passed
Zero/trivial	3	TC001,TC002,TC003	3/3
Impossible	4	TC010,TC011,TC012,TC013	4/4
Greedy Fails	3	TC020,TC021,TC022	3/3
Large Performance	5	TC040,TC041,TC042,TC043,TC044	4/5
Edge Cases	2	TC050,TC051	2/2

6.Conclusion

1)The algorithm works perfectly for the target amount being less than or equal to 10^8 .

2)The algorithm works perfectly fine even when denominations are repeated .

3) The algorithm gives error in impossible cases.

4) The algorithm works even in the cases where greedy algorithm fails. The algorithm is more accurate than greedy algorithm.

7. Appendices

7.1 Full implementation code

This shows the code for minimum coins , in python

```
File Edit Format Run Options Window Help
def cp(D,n):
    '''This is a function which takes two inputs, a
       set/list of denominations D and an amount n
       and returns the minimum number of coins required
       to get the amount n. It returns -1 if the amount
       can't be achieved'''
    try:
        m=min(D) #smallest denomination
        N=dict()
        '''This variable act as an array storing solution
           for cp(D,i) for i <= n, if not possible give infinity '''
        for i in range(n+1):
            if i==0: ## for zero amount no coins are required
                N[i]=0
            elif i<m: # if amount entered is less than smallest denomination gives infinity.
                N[i]=float('inf')
            else:
                S=set()
                for d in D:
                    if i>=d:# for i<d it is not possible to achieve i using d.
                        S.add(N[i-d])# S = {N(i-d)| d in D, i>=d}
                N[i]=1+min(S)# N[i] = 1+ min{N(i-d)| d in D, i>=d}
        if N[n]==float('inf'):
            return -1#if not possible give value -1
        return N[n]
    except:
        print("Error")
```

Test function give minimum coins required as well as time in seconds to execute.

```
File Edit Format Run Options Window Help
import time, Coin_Change

def test(D,n):
    start_time=time.perf_counter()
    result=Coin_Change.cp(D,n)
    end_time=time.perf_counter()
    run_time=end_time-start_time
    print(result,run_time,sep="\n")
```

7.2 Screenshot of output

```
*IDLE Shell 3.13.7*
File Edit Shell Debug Options Window Help
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:\Users\kaust\Downloads\Python\Test.py =====
>>> #Basic Cases
>>>
>>> test({1,2,5},0)#test ID TC001
0
1.2299999980314169e-05
>>>
>>> test({7},7)#test ID TC002
1
5.029999988437339e-05
>>>
>>> test({7,10,30},30)#test ID TC003
1
4.719999992630619e-05
>>>
>>> #Impossible Cases
>>>
>>> test({},5)#test ID TC010
Error
None
0.012287899999819274
>>>
>>> test({4,10},-2)#test ID TC011
Error
None
0.07641650000005029
>>>
>>> test({0,1,4},9)#test ID TC012
Error
None
0.07488120000016352
>>>
>>> test({-2,2,4},10)#test ID TC013
Error
None
0.057488200000079814
>>>
>>> #Cases Where Greedy Algorithm Fails
>>>
>>> test({1,4,9},12)#test ID TC020
3
3.620000006776536e-05
>>>
>>> test({7,9},14)#test ID TC021
```

25°C
Partly cloudy

Search

```
IDLE Shell 3.13.7
File Edit Shell Debug Options Window Help
>>> #Cases Where Greedy Algorithm Fails
>>>
>>> test([1,4,9],12)#test ID TC020
3
3.620000006776536e-05
>>>
>>> test([7,9],14)#test ID TC021
2
2.2499999886349542e-05
>>>
>>> test([27,30],81)#test ID TC022
3
0.00012289999995118706
>>>
>>> #Large Values/Performance Cases
>>>
>>> test([1,5,10,25],1000000)
40000
0.6146269000000757
>>>
>>> test([1,5,10,25],10000000)#test ID TC041
400000
6.5332281999999395
>>>
>>> test([1,5,10,25],100000000)
4000000
150.42387150000013
>>>
>>> test([1,5,10,25],500000000)#test ID TC043
Error
None
354.3032188999996
>>>
>>> test([100000,3000000,10000000,400000],100)#test ID TC044
-1
0.0005864999993718811
>>>
>>> #Edge/Boundary Cases
>>>
>>> test([1,2,2,5],7)#test ID TC050
2
7.249999998748535e-05
>>>
>>> test([10,12,12,20],34)#test ID TC051
3
5.1700000767596066e-05
>>>
```